

Light GBM

GOSS (Gradient-based One-Side Sampling)

- Solution to uniform sampling where you use 100% of observations $\Rightarrow$ SLOW
- Information gain is maximised by HIGH gradients.

Process:

- Sort all instances by $|\text{gradient}|$
- keep the top $a\%$ instances (large gradients)
- Randomly sample $b\%$ from remaining
- Amplify small gradients by a factor of $\frac{1-a}{b}$ to correct fact we dropped most.

EFB (Exclusive Feature Bundling)

Problem: real world data has thousands of sparse features (zero most of the time)

- Features rarely "conflict":

	cat	dog	fish	bird		Bundled
1	1	0	0	0	$\Rightarrow$	1
2	0	1	0	0		3
3	0	0	1	0		5
4	0	0	0	1		7

One-hot encoding

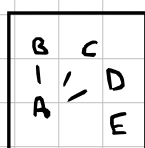
How to bundle?

Say we have 5 features A, B, C, D, E.

Two features can't be in same bundle if they are ever non-zero at the same time.

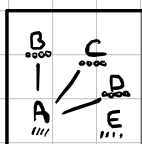
$\Rightarrow$ Reduces to graph-coloring problem.

Conflicts as graphs:



A & B conflict
A & C conflict
A & D conflict

Assign each feature a color, such that no two connected features share same color:



- {B, C, D}, never conflict
- {A, E}, never conflict

Histogram-based algorithm

Problem: pre-sorted algorithm is slow

Binning

For one feature, say "size in m^2 ":

Raw values: [23.4, 153.2, 56.8, 203.1, ...]

Bins: 0: [0, 50) $\rightarrow$ "small"

1: [50, 100) $\rightarrow$ "medium"

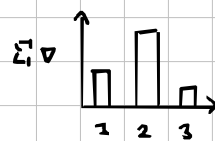
2: [100, 200) $\rightarrow$ "large"

Map every value to bin index:

[23.4, 153.2, ...]
 ↓ ↓
 0 1

Histogram

Sum gradients per bin to get histogram, on all observation



Complexity is $O(\# \text{data})$

Compute gain for each split.

$\hookrightarrow$ Complexity is $O(\# \text{bin}) \sim$ negligible.

Parameters

- num-leaves, default = 31, max number of leaves in one tree
- min-data-in-leaf, default = 20, minimal number of data in one leaf
- feature-fraction, default = 1 ($\in [0, 1]$), LightGBM will randomly select a subset of features.
 - $\hookrightarrow$ if < 1 , say 0.8, select randomly 80% of features
- bagging-fraction, default = 1, like feature-fraction but on part of data
 - $\hookrightarrow$ needs bagging-freq $\neq 0$
- bagging-freq, default = 0, frequency of bagging
 - $\hookrightarrow$ 0 = disabled, k = perform every k operations
- learning-rate, default = 0.1, shrinkage rate
- num-iterations (n-estimators), default = 100, number of boosting iterations
- lambda-1, default = 0, L1 regularization
- lambda-2, default = 0, L2 regularization

Tuning (Optuna)

- ① Define objective function
- ② Create study
- ③ Optimize

```
def objective(Trial):  
    params = {  
        "objective": "regression",  
        "metric": "rmse",  
        "learning-rate": Trial.suggest_float("learning-rate", 1e-3, 0.3, log=True),  
        "num-leaves": Trial.suggest_int("num-leaves", 2, 2**10)  
    }  
  
    model = lgb.LGBMRegressor(**params)  
    model.fit(X_train, y_train)  
    predictions = model.predict(X_val)  
    rmse = mean_squared_error(y_val, predictions, squared=False)  
    return rmse  
  
① Objective Function  
② study = optuna.create_study(direction="minimize")  
   study.optimize(objective, n_trials=20)  
③ print(study.best_params)  
   print(study.best_value)
```